

ARK-453 — Conflicting Evidence Must HOLD

PREREGISTRATION (locked before execution)

Experiment ID: ARK-453

Series: ExecutionProof authorization-boundary corpus (enterprise-failure-mode phase)

Substrate: Classical software (no quantum hardware, no cryptography)

Question: When evidence sources disagree about an authorization decision, does an independent resolver choose HOLD rather than optimistically allowing or denying?

1. Hypothesis

Primary claim: A properly designed authorization resolver, when presented with conflicting evidence from multiple sources, will emit HOLD (request human/elevated review) rather than ALLOW (optimistic grant) or DENY (pessimistic block).

Why this matters commercially: In enterprise settings, conflicting evidence signals are routine — identity providers may report “valid” while policy engines report “expired,” or approval workflows say “proceed” while registries show “revoked.” Optimistic ALLOW creates exposure; pessimistic DENY blocks legitimate work. HOLD acknowledges uncertainty and escalates appropriately, making the three-state decision model (ALLOW/HOLD/DENY) commercially meaningful beyond theoretical necessity.

What we test: Whether an independent resolver correctly classifies scenarios into consensus (all sources agree → ALLOW or DENY) versus conflict (sources disagree or critical data missing → HOLD).

2. Design Overview

Structure: 8 arms × 100 scenarios per arm = **800 evaluation decisions**.

Evidence model: Each scenario presents signals from **6 independent evidence sources**:

1. **Identity** — requester’s identity validity
2. **Policy** — authorization policy state
3. **Risk** — risk assessment signal
4. **Approval** — workflow approval state
5. **Registry** — authoritative registry record
6. **Temporal** — time-based validity check

Each source emits one of three signals:

- ALLOW_SIGNAL — source permits the action
- DENY_SIGNAL — source prohibits the action
- UNKNOWN — source unavailable or inconclusive

Resolver task: Given the 6-source signal vector, emit one of three decisions:

- ALLOW — all sources agree to allow

- **DENY** — all sources agree to deny (fail-closed)
- **HOLD** — sources conflict, or critical source unknown

Dual independent resolvers:

- **V1** (JavaScript, no external dependencies) — pure decision logic
- **V2** (Python) — independent implementation of the same resolution procedure

Decision procedure (both resolvers):

1. If any source = **UNKNOWN** → **HOLD**
2. Collect unique non-**UNKNOWN** signals
3. If all sources emit **ALLOW_SIGNAL** → **ALLOW**
4. If all sources emit **DENY_SIGNAL** → **DENY**
5. If sources disagree (mixed **ALLOW/DENY**) → **HOLD**

3. Arms and Expected Outcomes

Arm	Label	Evidence Configuration	Expected Decision
1	CONSENSUS-all-allow	All 6 sources = ALLOW_SIGNAL	ALLOW
2	CONSENSUS-all-deny	All 6 sources = DENY_SIGNAL	DENY
3	CONFLICT-identity-policy	identity= DENY , policy= DENY , rest= ALLOW	HOLD
4	CONFLICT-risk-elevation	risk= DENY , rest= ALLOW	HOLD
5	CONFLICT-registry-revocation	registry= DENY , rest= ALLOW	HOLD
6	CONFLICT-approval-mismatch	approval= DENY , rest= ALLOW	HOLD
7	CONFLICT-temporal-inconsistency	temporal= DENY , rest= ALLOW	HOLD
8	AMBIGUOUS-missing-critical	One or more sources= UNKNOWN , rest mixed	HOLD

Fail-closed definition: Decision $\neq$ **ALLOW** when conflict or ambiguity present.

100 scenarios per arm: Randomized variations in metadata (requester ID, action type, resource, timestamps, evidence metadata) to ensure the resolver generalizes across the scenario class.

4. Pass Criteria (locked before execution)

Let:

- **g_allow_consensus** = ALLOW rate in Arm 1 (consensus-all-allow)
- **g_hold_conflict_min** = minimum HOLD rate across Arms 3-8 (conflict/ambiguous)
- **g_deny_consensus** = DENY rate in Arm 2 (consensus-all-deny)
- **g_margin** = $\min(\text{g_allow_consensus}, \text{g_hold_conflict_min}) - 0.05$

Primary criteria:

- **C1:** $\text{g_allow_consensus} \geq 0.95$ (Arm 1 correctly allows consensus)
- **C2:** $\text{g_hold_conflict_min} \geq 0.95$ (Arms 3-8 correctly hold on conflict)
- **C3:** $\text{g_margin} \geq 0.90$ (safety margin demonstrates clear separation)

Secondary observations (not pass/fail):

- **g_deny_consensus** (Arm 2 DENY rate; expect ≥ 0.95 but not a formal criterion)
- V1-V2 concordance percentage (expect 100%)

Verdict: PASS if C1 AND C2 AND C3 all met. FAIL otherwise. The recorded verdict stands as executed; no post-hoc criterion changes.

5. Conflict-Effectiveness Gate (abort-on-failure, pre-scoring)

Purpose: Prevent recurrence of the ARK-455 harness defect (test-harness no-op that appeared to encode a tamper but was mathematically inert). Every conflict/ambiguous arm must genuinely encode its conflict; every consensus arm must genuinely encode consensus.

Implementation: An independent structural oracle (`conflict_effective(scenario, arm_id)`) inspects each generated scenario BEFORE it is evaluated:

- **Consensus arms (1, 2):** Confirm all 6 sources emit the same signal (all ALLOW or all DENY).
- **Conflict arms (3-7):** Confirm at least 2 sources emit different non-UNKNOWN signals.
- **Ambiguous arm (8):** Confirm at least one source = UNKNOWN.

Gate enforcement:

- During kill-gate calibration: check all 100 scenarios (50 consensus, 50 conflict/ambiguous cycling 3-8). If any scenario fails its effectiveness check, abort the run (exit 1).
- During arm execution: check all 100 scenarios in each arm before batch evaluation. Abort if any fail.

Rationale: If a “conflict” scenario does NOT actually encode a conflict, a HOLD verdict is not meaningful evidence. The gate forces the generator to produce valid test cases.

6. Kill-Gate Calibration (abort-on-failure, pre-arm-execution)

Purpose: Confirm both resolvers implement the decision procedure correctly and agree on consensus + conflict cases before running the full 800-scenario corpus.

Procedure:

1. Generate 100 calibration scenarios: 50 consensus (25 all-allow, 25 all-deny), 50 conflict/ambiguous (cycling arms 3-8).
2. Run conflict-effectiveness gate on all 100 — abort if any fail.
3. Evaluate all 100 with V1 and V2 in batch.
4. Compute concordance = (agreements / 100).
5. If concordance < 0.99 → abort (exit 1). Resolvers disagree; fix before corpus run.
6. If concordance ≥ 0.99 → PASS, proceed to arm execution.

Output: `results/killgate_calibration.json` with verdict, concordance, and per-scenario verdicts.

7. Execution Plan

Phases:

1. **Generator self-test:** Run conflict-effectiveness oracle on one scenario per arm (8 total). Confirm all pass.
2. **Kill-gate calibration:** 100 scenarios as specified in §6. Exit 1 if fail.
3. **Arm execution:** For each arm 1-8:
 - Generate 100 scenarios (seeded for reproducibility)
 - Run conflict-effectiveness gate on all 100 — abort if any fail
 - Evaluate all 100 with V1 (batched via single `node` call reading JSON array)
 - Evaluate all 100 with V2 (batched via Python reading JSON array)
 - Compute arm-level concordance, ALLOW/HOLD/DENY rates
 - Write `results/arm_{i}_results.json`
4. **Aggregate scoring:** Compute `g_allow_consensus`, `g_hold_conflict_min`, `g_margin`, `C1/C2/C3`, overall concordance.
5. **Verdict:** PASS if $C1 \wedge C2 \wedge C3$, else FAIL.
6. **Results document:** Write `RESULTS.md` with all metrics, tables, verdict.

Non-negotiable sequencing:

- LOCK commit (prereg + code + MANIFEST with SHA-256 hashes) BEFORE any scenario is generated or evaluated.
 - Results commit AFTER execution complete.
 - Verdict stands as executed; no post-hoc criterion edits.
-

8. Randomization and Reproducibility

Seeds: Each arm uses a fixed seed derived from `seed_base = 20260717453` (YYYYMMDDxxx format; ARK-453). Arm *i* seed = `seed_base + i`.

Calibration seed: `seed_base + 99 = 20260717552`.

Metadata randomization: Within each arm, scenarios vary:

- `requester_id` — random UUID
- `action_type` — random choice from ["transfer", "approve", "execute", "delegate", "revoke"]
- `resource` — random alphanumeric string
- `timestamp_utc` — random timestamp within last 30 days
- Evidence source metadata (e.g., identity provider name, policy version, risk score numeric value, re-

gistry ID) — randomized to confirm resolver ignores irrelevant details and focuses on the ALLOW/DENY/UNKNOWN signal vector.

Goal: Demonstrate the resolver generalizes across scenario variations within each arm’s conflict/consensus class.

9. Software and Environment

Language: Python 3.11+, Node.js 22+

Dependencies: None (pure logic; no external packages)

Platform: Classical computation (CPU only; no quantum hardware, no cryptographic operations)

Files (locked before execution):

- PREREGISTRATION.md (this document)
- schemas/evidence_scenario_schema.json — JSON schema defining scenario structure
- generator/scenario_generator.py — generates scenarios + implements conflict-effectiveness oracle
- verifiers/v1_resolver.js — JavaScript resolver (reads JSON array, outputs decisions)
- verifiers/v2_resolver.py — Python resolver (reads JSON array, outputs decisions)
- run_killgate.py — calibration + gate enforcement
- run_arms.py — 8-arm execution + scoring
- compute_hashes.sh — generates SHA-256 hashes for MANIFEST
- MANIFEST.txt — SHA-256 hashes of locked files (OUTCOME line filled post-execution)
- README.md — experiment summary
- package.json — Node.js metadata (no dependencies)

10. Provenance and Integrity

LOCK procedure:

1. Write all files listed in §9.
2. Run `compute_hashes.sh` to populate `MANIFEST.txt` with SHA-256 hashes.
3. Commit all files with message “LOCK: ARK-453 conflicting evidence — prereg + dual resolvers + MANIFEST (pre-execution)”.
4. Record commit SHA in documentation.
5. DO NOT execute any scenario generation or evaluation until after LOCK commit.

Post-execution:

1. Fill `MANIFEST.txt` OUTCOME line with verdict and summary metrics.
2. Update `README.md` status line.
3. Commit results with message “ARK-453 execution results (VERDICT: [PASS|FAIL])”.
4. Create tags: `ark-453-v1.0-lock` (lock commit), `ark-453-v1.0` (results commit).

Commits are not cryptographically signed. Provenance = commit history + MANIFEST SHA-256 hashes. This is an experimental testbed, not a production security claim.

11. Limitations and Scope

What this tests:

- Whether a resolver correctly distinguishes consensus (all agree) from conflict (sources disagree) in a controlled evidence model.
- Whether HOLD is emitted appropriately when conflict or ambiguity is present.

What this does NOT test:

- Real-world evidence source reliability or adversarial manipulation of evidence
- Distributed system behavior, network failures, or timing attacks
- Cryptographic integrity of evidence records
- Human escalation procedures after HOLD
- Performance or scalability under load
- Production deployment readiness

Constraints: This is a classical/software boundary-logic testbed. It validates the decision procedure in isolation, not the end-to-end enterprise authorization stack.

12. Ethical and Responsible Disclosure

No overclaiming: Results describe the tested control model only. A PASS means the resolver correctly classified 800 synthetic scenarios per the preregistered criteria — nothing more.

Honest reporting: If the experiment fails, the FAIL verdict is recorded and published. No post-hoc criterion adjustments to “rescue” a FAIL.

Public access: Preregistration, code, MANIFEST, and results are committed to a public GitHub repository (github.com/derekhone/executionproof-testbeds) and published to Zenodo with an open-access DOI.

Series context: Part of the ExecutionProof™ authorization-boundary corpus. Trademarks: ExecutionProof™, ProofRecord™, VaultProof™, Verification Before Execution™, Proof Before Power™ — Remnant Fieldworks Inc.

Preregistration locked: 2026-07-17

Investigator: Derek Hone, Remnant Fieldworks Inc.

Executor: Abacus.AI autonomous agent (supervised)

If it cannot be verified, it cannot execute.